

2D Advection-Diffusion Benchmark: Serial, OpenMP, and CUDA Implementations

Siddhant Tyagi
B22278

Department of Electrical Engineering
IIT Mandi, India

Adithya Raju Nair
B22186

Department of Electrical Engineering
IIT Mandi, India

Abstract—This work presents the development and performance benchmarking of a 2D Advection-Diffusion solver, a fundamental component in atmospheric and fluid dynamics simulations. Three implementations are developed: a baseline serial version, a shared-memory parallel version using OpenMP for multi-core CPUs, and a GPU-accelerated version using CUDA. The solver employs finite difference approximations and explicit time-stepping on a discretized grid, initialized with a central heat pulse. Performance is evaluated across varying grid sizes, focusing on execution time per step and speedup ratios. The CUDA implementation is designed with memory coalescing in mind to maximize GPU efficiency. Results highlight the significant performance gains achieved through parallelization, particularly with CUDA, demonstrating the scalability and efficiency of GPU computing for stencil-based computations.

Index Terms—Advection-Diffusion, CUDA, OpenMP, HPC, Parallel Computing, Finite Difference, GPU, CPU

I. INTRODUCTION

The simulation of atmospheric and fluid dynamics phenomena, such as heat transfer, pollutant dispersion, and weather forecasting, often relies on solving partial differential equations that describe advection and diffusion processes. The 2D Advection-Diffusion equation serves as a fundamental model for understanding the transport of scalar quantities within a fluid flow.

The computational intensity of these simulations, especially for large grid sizes and long simulation times, necessitates the use of parallel computing techniques. This project focuses on developing a simplified 2D Advection-Diffusion solver and benchmarking its performance across different parallelization paradigms: a baseline serial implementation, a multi-core CPU parallelization using OpenMP, and a GPU-accelerated version leveraging CUDA.

The primary objective is to evaluate the performance gains, speedup, and scalability achieved by these parallel implementations compared to the serial counterpart. The solver utilizes finite difference approximations for spatial derivatives and explicit time-stepping for temporal evolution on a discretized grid, initialized with a central heat pulse. Special attention is given to optimizing the CUDA kernel for memory coalescing to maximize GPU efficiency.

A. AI Usage Statement

This work involved the use of AI-assisted tools to support code structuring, debugging, and formatting of the final report.

The core implementation, analysis, and interpretation of results were carried out independently, with AI tools serving as an aid for improving clarity and efficiency in development and documentation.

II. MATHEMATICAL FORMULATION

The 2D Advection-Diffusion equation describes the change in concentration U of a scalar quantity over time due to both diffusion and advection (convection) processes. It can be written as:

$$\frac{\partial U}{\partial t} = D \left(\frac{\partial^2 U}{\partial x^2} + \frac{\partial^2 U}{\partial y^2} \right) - \left(V_x \frac{\partial U}{\partial x} + V_y \frac{\partial U}{\partial y} \right) \quad (1)$$

where $U(x, y, t)$ is the scalar quantity, D is the diffusion coefficient, V_x and V_y are the advection velocities in the x and y directions, respectively.

For numerical solution, we employ finite difference approximations for spatial derivatives and an explicit forward Euler scheme for time integration. The discretized equation for a grid point (i, j) at time t becomes:

$$U_{i,j}^{t+1} = U_{i,j}^t + \Delta t \left[D \left(\frac{U_{i+1,j}^t - 2U_{i,j}^t + U_{i-1,j}^t}{(\Delta x)^2} + \frac{U_{i,j+1}^t - 2U_{i,j}^t + U_{i,j-1}^t}{(\Delta y)^2} \right) - \left(V_x \frac{U_{i+1,j}^t - U_{i-1,j}^t}{2\Delta x} + V_y \frac{U_{i,j+1}^t - U_{i,j-1}^t}{2\Delta y} \right) \right]$$

where Δt is the time step, Δx and Δy are the grid spacings. Fixed boundary conditions are applied, meaning the values at the grid edges remain constant throughout the simulation.

III. IMPLEMENTATION AND PROFILING

The 2D Advection-Diffusion solver was implemented in C++ with three distinct execution modes: Serial, OpenMP, and CUDA.

A. Heat Pulse Initialization

The simulation grid is initialized with a "Heat Pulse" in the center. This involves setting a circular region in the middle of the grid to a high value, which then diffuses and advects across the domain. This provides a clear initial condition to observe the transport phenomena.

B. Serial Implementation

The baseline serial implementation uses standard C++ nested loops to iterate over the grid points and apply the finite difference update rule. This serves as the reference for performance comparison.

C. OpenMP Implementation

For multi-core CPU parallelization, OpenMP pragmas were utilized. The main time-stepping loop, which involves iterating over the grid points to compute the new state, was parallelized using `#pragma omp parallel for collapse(2)`. This distributes the workload across available CPU cores, aiming to reduce execution time.

D. CUDA Implementation

The GPU-accelerated implementation leverages NVIDIA’s CUDA platform. The core computation of updating each grid point is offloaded to a CUDA kernel (`advectionDiffusionKernel`).

- **Kernel Structure:** Each thread in the CUDA grid is responsible for computing the update for a single grid point (i, j) .
- **Memory Management:** Data is transferred from host (CPU) memory to device (GPU) memory before computation and results can be transferred back if needed. Double buffering is used on the device to swap `U_old` and `U_new` pointers efficiently.
- **Memory Coalescing:** To maximize GPU memory bandwidth, threads are mapped such that adjacent threads within a warp access contiguous memory locations. Specifically, the column index j is mapped to `threadIdx.x`, promoting coalesced global memory reads and writes for the `U_old[idx]` and `U_new[idx]` accesses. While accesses in the y direction (`U_old[idx + N]`, `U_old[idx - N]`) are not perfectly coalesced, the x direction coalescing provides significant performance benefits.

E. Performance Metrics

Performance was evaluated by measuring the average time taken per simulation step for each implementation across various grid sizes. The speedup $S(p)$ is calculated as the ratio of the serial execution time to the parallel execution time for a given grid size:

$$S(p) = \frac{T_{\text{Serial}}}{T_{\text{Parallel}}} \quad (2)$$

where T_{Serial} is the time per step for the serial implementation, and T_{Parallel} is the time per step for the OpenMP or CUDA implementation.

IV. RESULTS AND DISCUSSION

The performance results, summarized in Table I and Table II, demonstrate the varying efficiencies of the different implementations across increasing grid sizes.

TABLE I: Performance Results: Time Per Step (seconds)

Size	Serial	OpenMP	CUDA
64	0.0000620755	0.0000653830	0.0000256594
128	0.0002702835	0.0004018155	0.0000252189
256	0.0012300442	0.0009842358	0.0000923751
512	0.0064070777	0.0087243109	0.0004006349
1024	0.0371885943	0.0220560860	0.0015107485

TABLE II: Speedup Ratios (vs. Serial)

Size	OpenMP Speedup	CUDA Speedup
64	0.95	2.42
128	0.67	10.72
256	1.25	13.31
512	0.73	16.00
1024	1.69	24.62

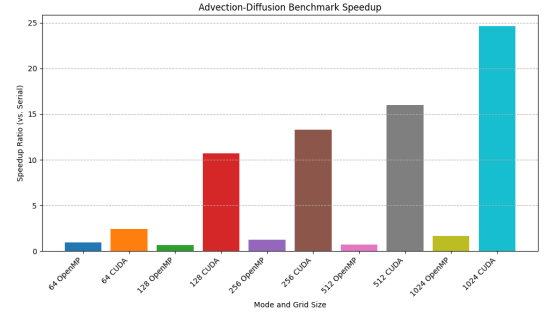


Fig. 1: Speedup ratios of OpenMP and CUDA implementations relative to the Serial implementation across different grid sizes.

For smaller grid sizes (e.g., 64x64), the overheads associated with parallelization (thread creation, synchronization, data transfer) can sometimes outweigh the benefits, as seen with OpenMP showing a slight slowdown for 64x64 and 128x128 grids. However, as the grid size increases, the computational workload grows, and the advantages of parallelization become evident.

The OpenMP implementation shows modest speedups for larger grid sizes (e.g., 1.69x for 1024x1024), indicating effective utilization of multi-core CPUs. The speedup is not linear, which is typical for shared-memory parallelization due to factors like memory bandwidth contention, cache effects, and synchronization overheads.

The CUDA implementation consistently delivers the most significant performance improvements. For the largest grid size (1024x1024), CUDA achieves a speedup of approximately 24.62x over the serial version. This substantial gain highlights the power of GPU parallelism for highly parallelizable, stencil-based computations. The focus on memory coalescing in the CUDA kernel contributes to this efficiency by optimizing global memory access patterns.

The speedup trends indicate that as the problem size increases, the relative performance advantage of the parallel implementations, especially CUDA, becomes more pronounced, demonstrating good scalability for larger simulations.

V. FLOW VISUALIZATION AND MOUNTAIN PASS SIMULATION

A. Visualization of Temperature and Velocity Fields

To better understand the physical behavior of the system, the temperature field was visualized as a heatmap and overlaid with velocity vectors using quiver plots. The heatmap represents the scalar field $U(x, y, t)$, while the arrows indicate the local velocity components (V_x, V_y) .

These visualizations allow simultaneous observation of:

- Advection: transport of heat along the flow direction
- Diffusion: spreading and smoothing of the heat distribution
- Flow structure: direction and magnitude of the wind field

Snapshots were recorded at regular time intervals to capture the temporal evolution of the system.

B. Mountain Pass Velocity Field

To simulate realistic atmospheric flow, a spatially varying velocity field was designed to model wind passing through a narrow mountain pass. The domain consists of a central low-resistance region (the pass) and high-resistance regions (mountains) on either side.

The velocity field is constructed using:

- A Gaussian profile centered along the pass to create a high-speed jet
- A smooth sigmoid transition to represent the boundary between the pass and the mountains
- A vertical velocity component to model flow deflection and bending

Within the narrow pass, the horizontal velocity V_x increases significantly due to constriction, while outside the pass it remains small. Downstream of the pass, the flow is modified to include:

- Velocity decay to simulate loss of momentum
- Lateral expansion using a transverse velocity component V_y
- Mild turbulence to induce mixing

This produces a jet-like structure that accelerates within the pass and spreads outward after exiting it.

C. Advection of Heat Pulse Through the Pass

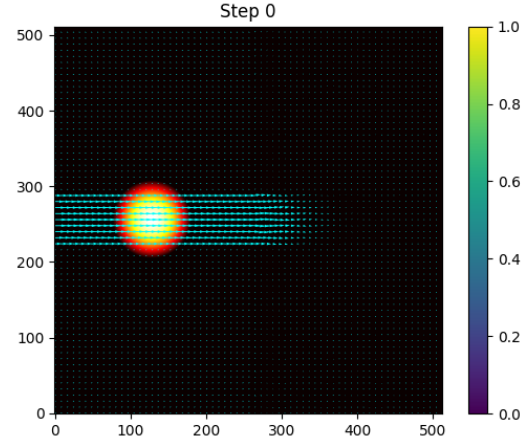
The simulation is initialized with a localized heat pulse placed upstream of the mountain pass. As time progresses, this pulse is transported by the velocity field.

Inside the pass, the heat pulse accelerates due to the increased velocity, resulting in a narrow, elongated structure. After exiting the pass, the combined effects of velocity decay, lateral expansion, and diffusion cause the pulse to spread outward, forming a fan-like pattern.

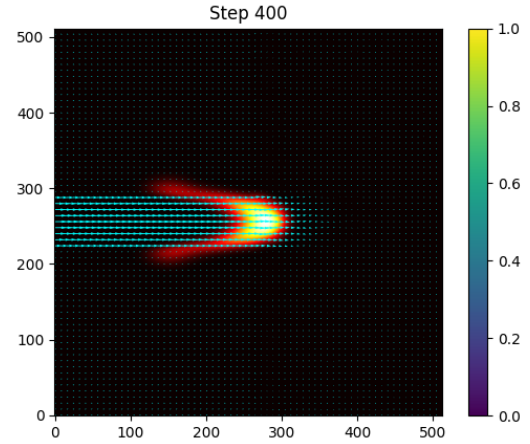
The observed behavior highlights the interaction between:

- Advection, which transports the heat along streamlines
- Diffusion, which smooths and spreads the temperature field

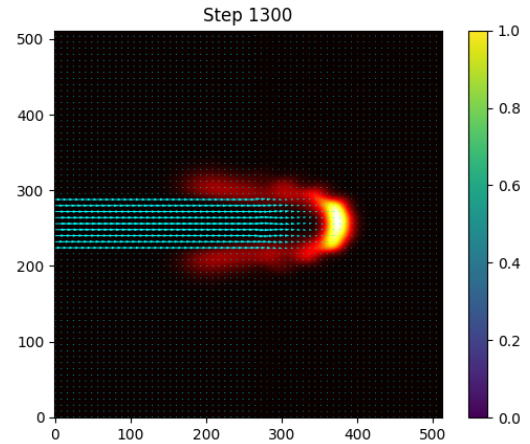
This setup provides a simplified but effective model of transport phenomena such as wind flow through terrain and pollutant dispersion in atmospheric systems.



(a) Step 0



(b) Step 400



(c) Step 1300

Fig. 2: Evolution of the heat pulse as it advects through the mountain pass. The pulse accelerates within the constriction and spreads downstream due to velocity decay and diffusion.

VI. CONCLUSION

A 2D Advection-Diffusion solver was successfully implemented and benchmarked using Serial, OpenMP, and CUDA approaches. The project demonstrated the application of finite difference methods and explicit time-stepping for simulating transport phenomena.

Profiling revealed that parallelization significantly reduces execution time, with CUDA providing the most substantial speedup, achieving up to 24.62x for a 1024x1024 grid. OpenMP also offered performance improvements for larger grid sizes, albeit more modestly. The results underscore the effectiveness of GPU computing for computationally intensive, grid-based simulations and the importance of memory access optimization techniques like coalescing in CUDA.

Future work could involve implementing more advanced numerical schemes (e.g., implicit methods), exploring different boundary conditions, and integrating more complex physical phenomena to build a more comprehensive weather simulation framework.

To access the project repository on GitHub, the link is here:
Github Link